# gen_smf_1d

## `gen_smf_1d.py` Simulation Guide

This file documents how `gen_smf_1d.py` runs the 1D SMF/BEC simulation, what physics it encodes, and how to run it.

## Physics Summary (SMF BEC Model)

This simulation implements the 1D single-mirror feedback (SMF) model of a BEC coupled to light:

- **BEC evolution:**

$$i\,\partial_t\psi = -\omega_r\nabla^2\psi + \frac{\Delta}{4}S(x,t)\,\psi$$

- **Optical field:**

$$F = \sqrt{p_0}\,e^{-i\frac{b_0}{2\Delta}|\psi|^2}, \quad B = \mathcal{F}^{-1}\left[\sqrt{R}\,\mathcal{F}(F)\,e^{-ik^2\,\mathrm{shift}}\right]$$

- **Feedback nonlinearity:**

$$S(x,t) = p_0 + |B|^2$$

This closed loop

$$\psi \to F \to B \to S \to \psi$$

generates the optomechanical instability and pattern formation.

## Purpose

`gen_smf_1d.py` evolves a 1D complex BEC field `psi(x,t)` and computes an optical intensity field `S(x,t)` from a self-consistent backward field `B`.

Outputs are written as time-series tables for each pump value `p0`.

## Physics/Model Used In The Code

The code uses a split-step style update each time-step:

1. **Kinetic propagation** in Fourier space:
   - `psi_k -> psi_k * exp(-i * omega_r * k^2 * dt)`
2. **Potential/nonlinear propagation** via RK2 ( `rk2`, `dy` ):

    - optical field amplitude: `F = sqrt(p0) * exp(-i * b0/(2*Delta) * |psi|^2)`
    - backward field from propagation kernel: `B = ifft( sqrt(R) * fft(F) * exp(-i*k^2*shift) )`
    - local evolution term: `dpsi/dt = -i * Delta/4 * (p0 + |B|^2) * psi`
3. The optical intensity written to file is:
    - `S = p0 + |B|^2`

It also includes optional seed/noise perturbations from `seed.in` ( `seed` , `noise1` , `noise2` , `noise3` ) and optional analytic delay-time-based simulation-time extension.

**Interpretation:**

- The kinetic step corresponds to dispersion / quantum pressure in the BEC.
- The nonlinear step implements the optical potential generated by the feedback field.
- The backward field `B` is what closes the feedback loop and enables pattern formation.

Most quantities in the simulation are expressed in **dimensionless/scaled units**, consistent with the theoretical model.

# Input Files

The script expects:

- `patt1d_inputs/<filename>/seed.in`

From `seed.in`, it reads (in order):

1. `nodes` — number of spatial grid points
2. `maxt` — total simulation time
3. `ht` — timestep size
4. `width_psi` — initial width of the condensate (Gaussian envelope)
5. baseline `p0` — pump saturation parameter
6. `Delta` — detuning (sets sign/strength of optical potential)
7. `omega_r` — recoil/kinetic coefficient (sets strength of dispersion)
8. `b0` — optical thickness (controls atom-light coupling strength)
9. `num_crit` — number of critical wavelengths in the domain
10. `R` — mirror reflectivity (feedback strength)
11. `gbar` — auxiliary parameter (used in analytic estimates, not directly in evolution)
12. `v0` — initial phase gradient (initial momentum of the BEC)
13. `plotnum` — number of output snapshots
14. `seed` — deterministic cosine perturbation amplitude

15. `noise1` — initial random noise amplitude
16. `noise2` — time-dependent noise amplitude
17. `noise3` — currently unused in evolution

### Additional notes on parameters

- The spatial domain is defined as
  `L_dom = 2π * num_crit`, so increasing `num_crit` increases system size.
- `omega_r` plays a key physical role:
  it controls how strongly the wavefunction spreads and therefore sets the scale of the **quantum suppression of small-scale structure**.
- `p0`, `b0`, and `R` together determine how strong the feedback-driven instability is.
- The sign of `Delta` determines whether atoms move towards:
  - high intensity (red detuning)
  - low intensity (blue detuning)

## Output Files

Outputs are written to:

- `patt1d_outputs/<filename>/psi...out`
- `patt1d_outputs/<filename>/s...out`

Each output row is:

- column 1: time `t`
- remaining columns: spatial values

Where:

- `psi` output stores `|psi(x,t)|^2` (density)
- `s` output stores `S(x,t) = p0 + |B|^2`

File naming:

- Single frame / default: `psi.out`, `s.out` style base names
- Sweeps / indexed runs: `psi<index>_<p0>.out`, `s<index>_<p0>.out`

## CLI Arguments

Required:

- `-f, --filename`

- `-n, --num_pump_frames`

Optional sweep control:

- `-s, --start_pump_param`
- `-e, --end_pump_param`
- `-i, --index` (useful for array jobs)

Optional non-uniform sampling in `p0`:

- `--density-centers <p0...>` (up to 2 centers)
- `--density-width <sigma...>`
- `--density-strength <value>`

These allow you to concentrate sampling near specific pump values (e.g. near threshold).

Optional time extension:

- `--extend-time-using-t0`
  - computes `p_th` and analytic `t0`
  - extends `maxt` up to `TIME_EXTENSION_FACTOR * t0` (capped by `MAX_ANALYTIC_TIME`)

This is useful when studying **delay times near threshold**, where dynamics become slow.

## Common Run Examples

Single simulation at `p0` from `seed.in`:

```
python gen_smf_1d.py -f <sim_filename> -n 1
```

```
python gen_smf_1d.py \
  -f <sim_filename> \
  -s 2.0e-10 -e 4.0e-10 \
  -n 21
```